CS 4290/6290

Multiprocessing and Multithreading

Portions of these notes are from Tom Conte, Josep Torrellas and Milos Prvulovic,

Copyright © 2012: These notes MAY NOT BE REPRODUCED WITHOUT EXPRESS PERMISSION OF THE AUTHORS

1

Multiprocessors

- Why do we need multiprocessors?
  - Uniprocessor speed still improving (slowly)
  - Days of doubling performance every 18 months are long gone
- Multiprocessor software problem
  - Grain size: bit-level, ILP, thread-level, program level (throughput)
  - Most code is sequential (for uniprocessors)
    - MUCH easier to write and debug
  - Correct parallel code very, very difficult to write
    - Efficient and correct is even harder
    - Debugging even more difficult (Heisenbugs)

MP-2

2

Multiprocessor Execution Models

Flynn's Taxonomy (re-review):

- Defines the "execution model" for a program
- SISD: Single I Stream, Single D Stream
  - A uniprocessor
- SIMD: Single I, Multiple D Streams
  - Each "processor" works on its own data
  - But all execute the same instrs in lockstep
  - E.g. a vector processor or MMX

MP-3

3

Flynn's Taxonomy: Execution models (cont.)

- MISD: Multiple I, Single D Stream
  - Not common: used for cryptography
  - Stream processor of IBM Project STRETCH was an MISD
- MIMD: Multiple I, Multiple D Streams
  - Each processor executes its own instructions and operates on its own data
- NEW: "Hybrid": SPMD: Single *program,* multiple data streams
  - Each thread can take different paths, but running the same code

MP-4

4

Taxonomy of Multiprocessors

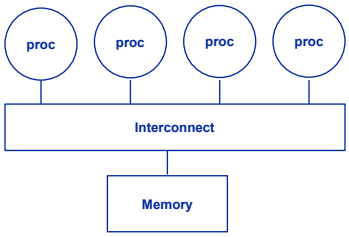
1. Physical location of memory
2. Type of Interconnect
3. Memory access latency variability
4. Programmer's memory (access) model

MP-5

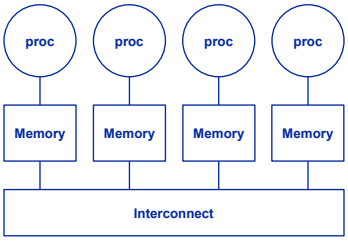
5

Physical Memory Location Types

Centralized Memory



Distributed Memory

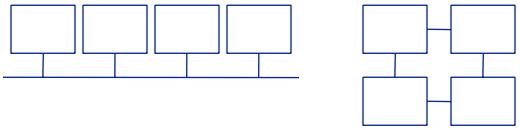


MP-6

6

Types of interconnects

- Shared common medium (e.g, BUS) vs. point to point



- Note: broadcast/multicast possible in both systems
- But shared common media allow for *observable* transactions
  - As stated before, for electrical property reasons, shared does not scale

MP-7

7

Memory Access Latency Types

1. "Symmetric Multiprocessors" (SMP) when interconnect is a bus

   "Uniform Memory Access" (UMA) is more general term for SMP
   - All memory locations have the same latencies (really, "similar latencies"…)
   - Problem: Memory Contention
     - All processor share the one memory thus memory bandwidth becomes a bottleneck
   - UMA is used only for smaller multiprocessors: 2,4, or 8 processors
2. Non-Uniform Memory Access

   Anything that isn't UMA!

MP-8

8

Programmer's memory models: *Message Passing* *vs.* *Shared Memory*

MP-9

9

Programmer Memory Models

1. Shared Memory
   - All threads on all processors see the same address space
   - If processor x writes to address A, processor y will (eventually) see the value at address A.
2. Non-shared memory: Message Passing
   - Each processor has its own address space
   - The only way to communicate is via (programmer specified) sends/receives

MP-10

10

Message-Passing Memory Model

- Producer-Consumer Scenario:
  - P1 produces data D, uses a SEND(P2, D) to send it to P2
  - The network routes the message to P2
  - P2 then calls a RECEIVE(P1) to get the message
- Two types of send primitives
  - Synchronous: P1 stops until P2 confirms receipt of message
  - Asynchronous: P1 sends its message and continues
- Standard libraries for message passing: Most common is MPI – Message Passing Interface

MP-11

11

Message Passing: A Program

- Calculating the sum of array elements

```
// Array is 1024 total size
// 4 processors
double A[64];
double mySum=0;
for(int i=0;i<64;i++)
  mySum+=A[i];
if(myPID=0){
  for(int p=1;p<4;p++){
    int pSum;
    recv(p,pSum);
    mySum+=pSum;
  }
  printf("Sum: %lf\n",mySum);
}else
  send(0,mySum);
```

Must manually split the array

"Master" processor adds up partial sums and prints the result

"Slave" processors send their partial results to master

MP-12

12

Shared Memory: A Program

- Calculating the sum of array elements

```
shared double A[1024];
shared double Sum=0;
shared mutex Lock;
double mySum=0;
for(int i=myPID*64;i<(myPID+1)*64;i++)
  mySum+=A[i];
lock(Lock);
Sum+=mySum;
unlock(Lock);
if(myPID=0)
  printf("Sum: %lf\n",Sum);
```

Array is shared

Each processor sums up "its" part of the array

Each processor adds its partial sums to the final result

"Master" processor prints the result

MP-13

13

Hardware for Shared Memory vs. Message

- Message passing…
  - Typically uses "off-the-shelf" machines
    - Not many new architectural concepts inside each machine
  - Mostly interconnection network issues
    - These can get pretty interesting
  - Programming is difficult
  - Synchronization is part of send/receive (i.e., receive waits for message)
- Shared memory needs architectural support
  - Plenty of interesting stuff to talk about here
  - Easier to program, but need synchronization instructions (i.e., locks and barriers)
- Multi-core almost always uses shared memory
- Thus, next several lectures on shared-memory

14

Hybrid: Distributed Shared Memory Machines

- Shared memory programming memory model *does not require* physical centralized memory model!
- "Distributed Shared-Memory (DSM)"
  - Add shared memory to message passing machines (with software or hardware or a combination)
  - All processors can address all memory locations
  - Latencies to access different memory locations can differ *a lot*
  - Also called NUMA (non-uniform memory access)

MP-15

15

Review

- Multiprocessor execution models
  - MIMD or SIMD
- Multiprocessor memory models
  - Physical: centralized vs. distributed
  - Programmer: Message passing vs. shared

MP-16

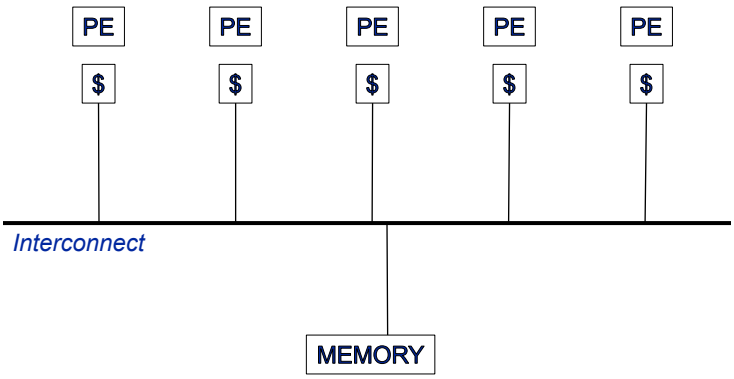
16

Shared Memory: Cache Coherence

MP-17

17

Centralized Shared Memory



MP-18

18

Cache Coherence Problem

- Shared memory easy with no caches
  - PE1 writes, PE2 can read
  - Only one copy of data exists (in memory)
- Caches store their own copies of the data
  - Those copies can easily get inconsistent
  - Classic example: adding to a sum
    - PE1 loads allSum, adds its mySum, stores new allSum
    - PE1's cache now has dirty data, but memory not updated
    - PE2 loads allSum from memory, adds its mySum, stores allSum
    - PE2's cache also has dirty data
    - Eventually PE1 and PE2's cached data will go to memory
    - Regardless of write-back order, the final value ends up wrong

MP-19

19

Cache Coherence Definition

- A memory system is coherent if
  1. **Read what is written**: A read of X on PE1 returns the value written by the most recent write to X on PE1 *if no other processor has written to X in between*.
  2. **Coherent writes happen *eventually***: If PE1 writes to X and PE2 reads X *after a sufficient time*, and there are no other writes to X in between, PE2's read returns the value written by PE1's write.
  3. **Causality of writes**: Writes to the same location are serialized: two writes to location X are seen in the same order by all PEs.

MP-20

20

Cache Coherence Definition: Why?

- Property 1. preserves program order
  - If no sharing, each PE acts like a uniprocessor
- Property 2. says that any write to an address must eventually be seen by all PEs
  - If PE1 writes to X and P2 keeps reading X, PE2 must eventually see the new value
- Property 3. preserves causality
  - Let X start = 0. PE1 sets X to 1. PE2 sets X to 2. All PEs see (X = 1, then X=2) OR (X=2, then X=1)
  - All PEs should see same final value for X

MP-21

21

Cache Coherence Definition

- Property 1: A processor can read what it wrote

PE1: Write X <= A
.
. *no other PE writes to X*
.
PE1: Read X, X == A

Why good? Each processor still a uniprocessor

MP-22

22

Cache Coherence Definition

- **All writes are globally seen eventually:**

PE1: Write X <= A

.
. *sufficient amount of time*
.

PE2: Read X, X == A

- **Why good? Guarantees values are propagated**

MP-23

23

Cache Coherence Definition

- **All writes are ordered globally in the system:**

PE1: Write X <= A

.
. *sufficient amount of time*
.

PE2: Read X = A PE3: Read X = A

.
.
.

PE2: Write X <= B

.
. *sufficient amount of time*
.

PE1: Read X = B PE3: Read X = B

- **Why good? Preserves causality**

MP-24

24

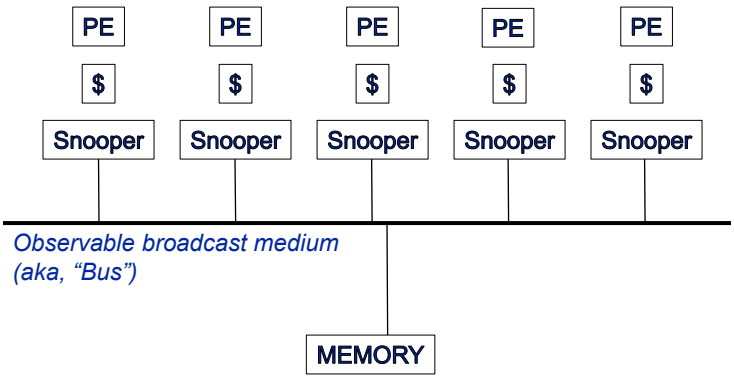
Schemes for Cache Coherence

- Shared Caches
  - Trivially enforces coherence
  - Not scalable– must be the only level of caching
- Snooping
  - Needs a broadcast network (like a bus) to enforce coherence
  - Each cache that has a block tracks its sharing state on its own
- Directory-based
  - Can enforce coherence even with a point-to-point network
  - A block has just one place where its full sharing state is kept

MP-25

25

System view for Snoopy Coherence



MP-26

26

Example: MSI Snoopy Protocol

- State of block B in cache $ can be:
  - Invalid: B is not cached in $
    - To read or write, must make a request on the bus
  - Modified: B is dirty in $
    - $ has the block, no other cache has the block, and $ must update memory when it displaces B
    - Can read or write B without going to the bus
  - Shared: B is clean in $
    - $ has the block, other caches have the block, and C need not update memory when it replaces B
    - Can read B without going to bus
    - To write, must send an upgrade request to the bus

MP-27

27

Example: MSI Snoopy Protocol

- State of block B in cache $ can be:

	My copy	*Can others have it?*	*Memory's copy*	*Notes*
Modified	dirty	none	stale	must write back on replacement
Shared	correct, clean	possibly	correct	can discard on replacement
Invalid	invalid	unknown	unknown	

MP-28

28

Bus actions

- A PE's cache coherence requests can be:
  - GetS – Read miss usually, this is a "get with intent to share"
  - GetM – This is a "read, may modify," meaning after I'm done, the block may be dirty (note: sometimes called GetX, the two mean the same thing)
- Snooper bus actions can be:
  - Abort – This is when the snooper needs a requestor to retry their request later
  - Intervene – This means the snooper will respond to a request instead of the memory

MP-29

29

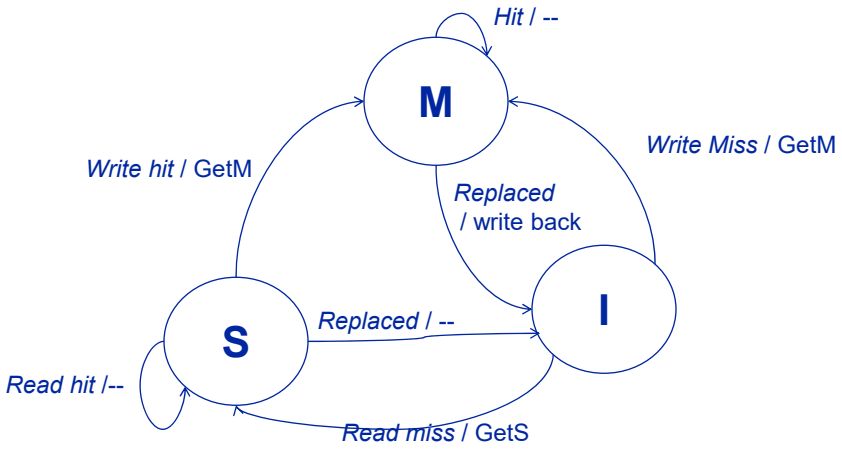
Cache to Cache transfers: Abort vs. Intervene

- Problem
  - PE1 has block B in M state
  - PE2 wants to read B, sends a GetS on bus
  - If PE1 does nothing, memory will supply the data to PE2
  - What does PE1 do?
- Solution 1: abort/retry
  - PE1 issues an ABORT, issues a write back
  - PE2 later retries GetS and gets data from memory
  - Too slow (two memory latencies to move data from PE1 to PE2)
- Solution 2: intervention
  - PE1 indicates it will supply the data ("INTERVENE" bus signal)
  - Memory sees that, does not supply the data, and waits for PE1's data
  - PE1 starts sending the data on the bus, memory is updated
  - PE2 snoops the transfer during the write-back and gets the block

MP-30

30

MSI state diagram: PE's $ perspective

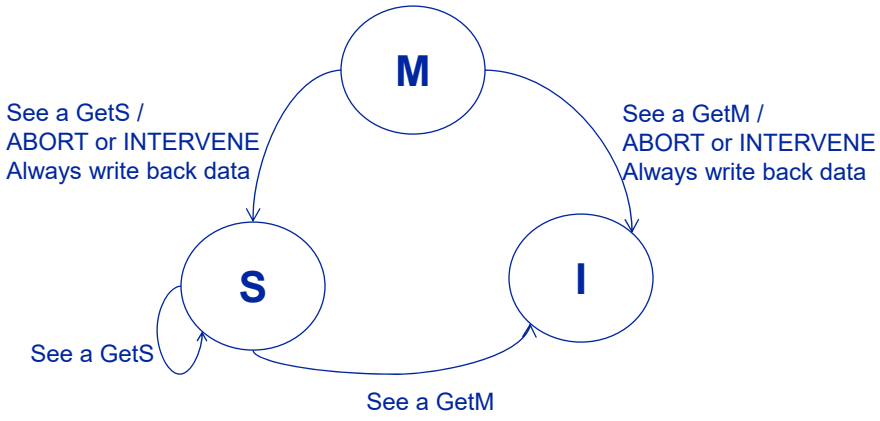


- **Legend:**
  $ state / PE's request

MP-31

31

MSI state diagram: Snooper's perspective



- **Legend:**
  Observed request / Bus action

MP-32

32

Example

PE1			PE2		
Condition	*Bus action*	*Start → end state*	*Condition*	*Bus action*	*state*
Read miss	GetS	I → S			I → I
		S → S	Read miss	GetS	I → S
Write hit	GetM	S → M	Invalidate		S → I
			Write miss	GetM	I → IM*
	ABORT Write back	M → I		Retry	IM* → M

*Note: IM is a transient state, means "transitioning from I to M"

MP-33

33

Other states: The Advantage of Exclusive

- Situation: PE1 makes a GetS request
- NO ONE ELSE makes a GetS request
- Next, PE1 wants to modify the data
  - Normally: Broadcasts a GetM … but why? No one else cares!
- Solution: "Exclusive," A state that means PE1 has the only, clean copy besides memory
  - Allows PE1 to "silently upgrade" to M

MP-34

34

Cache-To-Cache Transfers, part 2

- Intervention works if a cache has data in M or E state
  - Nobody else has correct data, so it's clear which PE is supposed to respond
  - But after 1 transfer, PE can no longer supply data (because it transitions to S)
- What if a cache has requested data in S state and others have it in S, which PE should supply the data?
  - Solution 1: let memory supply the data ☹
  - Solution 2: whoever wins arbitration supplies the data (interconnect must be a bus)
  - Solution 3: OWNER / FORWARDER – Indicates the PE is the transfer agent and supplies data. The difference: O is for dirty data, F is for clean data.

MP-35

35

How to Detect Other Sharers

- Needed for E-state, O-state or F-state
  - PE1 wants to read B, puts GetS on bus, receives data
  - How does PE1 know if its B should be in S or E state?
- Solution: "HIT" line
  - Always 0, except when a snooper from another Proc pulls it to 1 (called "hardwired or" by CompEs)
  - When, e.g., PE2 snoops PE1's request, if PE2 has the block in its cache, it pulls "HIT line" to 1
  - PE1 to S if "HIT" is 1, to E if "HIT" is 0

MP-36

36

The state of things: M,O,E,S,I,F

	My copy	*Can others have it?*	*Memory's copy*	*$ to $ transfer agent?*	*Responsibility to write back?*	*Notes*
Modified	dirty	Not possible	stale	1 time	Yes	can $ to $ once only
Owned	dirty	Possible (any others are in S state)	stale	Yes	Yes	
Exclusive	clean	Not possible	correct	1 time	No	*allows for silent upgrade to M or downgrade to I; can* $ to $ once only
Shared	clean	Possible	correct or unknown*	No	No	(*Mem state is **unknown** if **O** is implemented in protocol)
Invalid	invalid	Possible	unknown	No	No	
Forwarder	clean	Possible (any others are in S state)	correct	Yes	No	This is like the O state, but does not require writeback on replacement

MP-37

37

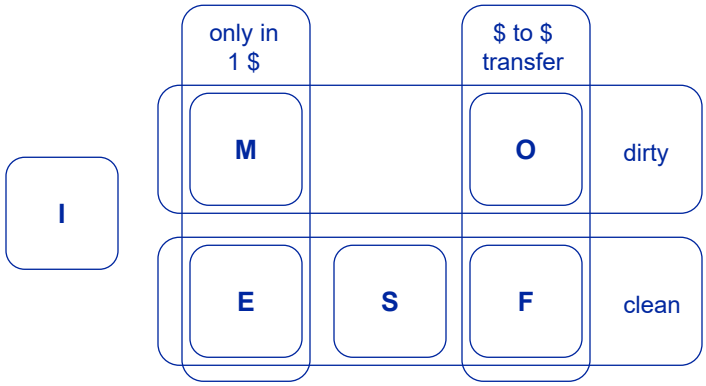
Other notes

- E is good because of the silent upgrade to M
- O, F (and write-update policies) are good if there is a long latency penalty to access memory
- In MOESIF, Can only get to O from M, can only get to F from E

MP-38

38

Another view



Permission strength for a write-invalidate protocol is:

M > E > O > F > S > I

or cache is allowed to…

M: read or write > E: read but silent upgrade on write > O: dirty but read-only, $-$ transfer agent > F: read-only, $-$ transfer agent> S: read only > I: none

MP-39

39

States vs. Protocols

- States != Protocols
- States are the "destination," protocol defines "how to get there"
- A protocol can support some state=>state transitions but not others
  - Example:

  M => O on an observed GetS,

  instead of M => S

  This is the choice of the protocol designer

MP-40

40

Kinds of sharing patterns

- Private: never shared
- Read-only: Data never written to, so always in state S (e.g., program constants)
- Migratory: Modified data moves from core to core, but not shared (e.g., the variable in a critical section)
- Producer/Consumer: PEx writes the data, PEy reads it (special case of migratory)
- Write-shared: Multiple writes by many PEs (as opposed to 1 write by a particular PE)
- General: Everything else ☺

MP-41

41

False sharing

- Protocols are optimized for certain sharing patterns
- Granularity of sharing can cause a problem…
  - Two variables, Q and R are stored in the same block-sized region of memory
  - Q and R have different sharing patterns
  - Protocols fail to work effectively
  - Called “false sharing”
  - Gets worse as block size is increased

MP-42

42

Coherence write policies

- Write-Update (broadcast on writes)
  - A write to shared data is broadcast so as to update all copies
  - All subsequent reads will return the new written value
  - All see the writes in the order of broadcasts One bus == one order seen by all
  - Difficult to verify implementation correctness!
- Write-Invalidate [most common]
  - Write to shared data forces invalidation of all other cached copies
  - Subsequent reads miss and fetch new value
  - Writes ordered by invalidations on the bus

MP-43

43

So which one? Write-Update vs. -Invalidate

- A burst of writes by a processor to one address
  - Write-Update: each write sends an update
  - Write-Invalidate: possibly only the first invalidation is needed
- Writes to different words of a block (false sharing)
  - W-Update: update sent for each word
  - W-Invalidate: possibly only the first invalidation is needed
- Producer-consumer sharing pattern
  - W-Update: producer sends an update, consumer reads new value from its cache
  - W-Invalidate: producer invalidates consumer's copy, consumer's read misses and has to request the block
- Which is better depends on application
  - But write-invalidate is simpler and implemented in most MP-capable processors today

MP-44

44

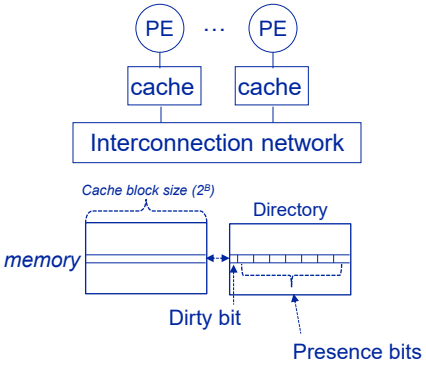
Directory Based Cache Coherence

- The BIG PROBLEM: what if there is no shared broadcast medium (no bus)?
  - Can't Snoop
  - Can't even guarantee access ordering!
- Key idea: track in a ***global directory*** (in main memory) which processors are caching a block, and the state of the block – DIRECTORY COHERENCE
  - Avoid broadcasts by:
    1. keeping track of ALL PEs caching a memory block
    2. using point-to-point messages to maintain coherence
  - Can use ANY scalable point-to-point network

MP-46

46

Basic (OLDEST) Scheme (Censier and Feautrier, 1978)



- Assume *P* PEs
- Each cache-block-sized region in memory (=2^B bytes wide) has $\lceil \log_2 P \rceil$ presence bits and 1 dirty bit,

MP-47

47

States in C&F directories

D	P0	P1	P2	P3	P4	P5	P6	P7
0	1	1	0	1	0	0	0	0

P0, P1 and P3 = S
All others = I

D	P0	P1	P2	P3	P4	P5	P6	P7
1	0	1	0	0	0	0	0	0

P1 = M
All others = I

D	P0	P1	P2	P3	P4	P5	P6	P7
0	1	0	0	0	0	0	0	0

P0 = E
All others = I

MP-48

48

Adding O/F to C&F

Add: index of O/F PE, valid bit for index

D	P0	P1	P2	P3	P4	P5	P6	P7	O/F id	Id valid
0	1	1	0	1	0	0	0	0	xxx	0

P0, P1 and P3 = S
All others = I

D	P0	P1	P2	P3	P4	P5	P6	P7	O/F id	Id valid
0	1	1	0	1	0	0	0	0	3	1

P0, P1= S
P3 = F, All others = I

D	P0	P1	P2	P3	P4	P5	P6	P7	O/F id	Id valid
1	1	1	0	1	0	0	0	0	3	1

P0, P1= S
P3 = O, All others = I

MP-49

49

Messages for MSI for C&F

Messages from directory to proc	*Meaning*
recall_goto_S(P, X)	P must send the directory the data for block X and then P's X goes to S.
recall_and_invalidate(P, X)	P must sends the directory the data for block X and P's X goes to I
invalidate(P, X)	P invalidates its data, sends ACK to directory
send(P, X)	P receives data X
sendACK(P, X)	P receives an ACK from dir. for X
Messages from proc to directory	***Meaning***
GetM(directory, X)	Get X from directory with intent to modify, Proc waits for ACK
GetS(directory, X)	Get X from memory read-only
Ack(dir, X)	Send directory an ACK for X

MP-50

50

MSI: Read Miss to X by *P*: *GetS(dir, X)*

```
if (X.state.dirty) {
        // Assume Q such that X.state[Q] == 1;
        memory[X] = recall_goto_S(Q, X);
        X.state[Q] = 1;       // not really needed
        send(P, memory[X]);
        X.state[P] = 1;
        X.state.dirty = 0;
    } else {
        X.state[P] = 1;
        send(P, memory[X]);
    }
```

MP-51

51

MSI: Write Miss of X by P: GetM(*dir, X*)

```
if (X.state.dirty) {
        // Assume Q such that X.state[Q] == 1;
        memory[X] = recall_and_invalidate(Q, X)
        X.state[Q] = 0;
        send(P, memory[X]);
        X.state[P] = 1;
} else {
        for each Q where X.state[Q] == 1 do {
                invalidate(Q, X)
                X.state[Q] = 0; }
        wait for all sharers to send ack messages
        send(P, memory[X]); // Note also sends ACK to P
        X.state[P] = 1;
        X.state.dirty = 1;
}
```

MP-52

52

MSI: Write Hit to X for *P and* X is in S*: GetM(dir, P)*

```
for each Q != P where X.state[Q] == 1 do {
        invalidate(Q, X)
        X.state[Q] = 0; }
// wait for all sharers to send ack messages
sendACK(P, X);
X.state[P] = 1;
X.state.dirty = 1;
```

MP-53

53

Key Issues/problems

- Directory becomes the bottleneck!
  - Need a distributed directory
- Overhead for state bits
  - Scales with the size of memory and the size of directory (ie, the # of processors)

  Dir size = (P + stateinfo) x ($M/2^B$)

  Where P = num processors,

  stateinfo= related to the protocol states (MOESIF),

  M is size of memory, and 2^B is the block size of the cache
- About "stateinfo" …
  - MESI = 1 (dirty bit)
  - MOESIF = 1 (*dirty bit*) + 1 (for *O/F*) + $\lceil \log_2 P \rceil$

    (index to store which of P procs has block in O/F plus valid bit for this index)

MP-54

54

Reducing storage overhead

- Overhead not too bad for medium-scale multiprocessors
  - Example: 30 Procs with 4GB of memory and with 64-byte cache blocks and MOESIF protocol ---> 4% memory overhead
- One way to reduce the storage overhead is to increase blocksize (2^B)
  - results in false sharing
  - increased coherence traffic
- Change dir entry to limit sharers (non-"full map" way)…

MP-55

55

Limited Pointer Schemes

- Since data is expected to be in only a few caches at any one time, a limited number of pointers per directory entry should suffice
- Overflow strategy: what to do when the number of sharers exceeds the number of pointers?
- Many different schemes based on different overflow strategies
- Limited pointers have Proc "ids" or "pointers," instead of per-Proc bit vectors

MP-56

56

Some Examples

- Dir-i-B (Broadcast) i "pointers" (ids), 1 invalidate-broadcast bit
  - Beyond *i* pointers, set the invalidate-broadcast bit ON
  - Storage needed per entry (see C&F state examples)
    - = i * $\lceil \log_2 P \rceil$ // *i Procs and need a valid per id*
    - + 1 + 1 // *Dirty bit + invalid broadcast bit*
    - + $\lceil \log_2 P \rceil$ + 1 // *for O, F, need id plus valid bit*
  - Expected to do well since widely shared data is not written often
- Dir-i-NB (No Broadcast)
  - When sharers exceed i, ***randomly*** invalidate one of the existing sharers
  - Significant degradation expected for widely-shared mostly-read data

MP-57

57

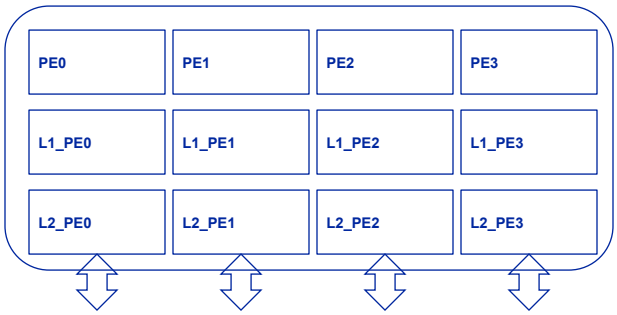
More…

- Dir-i-CV-r ("coarse vector": like multi-cast)
  - When sharers exceed i, use bits allocated to i "pointers" (ids) as a coarse resolution vector
    - e.g., say there are 256 procs and 2 pointers, that's 16 bits worth of data: 256/16 = 16 procs per coarse group, 1 in a group means all procs in that group may have the block
  - Always results in less coherence traffic than Dir-i-B

MP-58

58

Chip Multicore tradeoffs "Last Level Private" caching



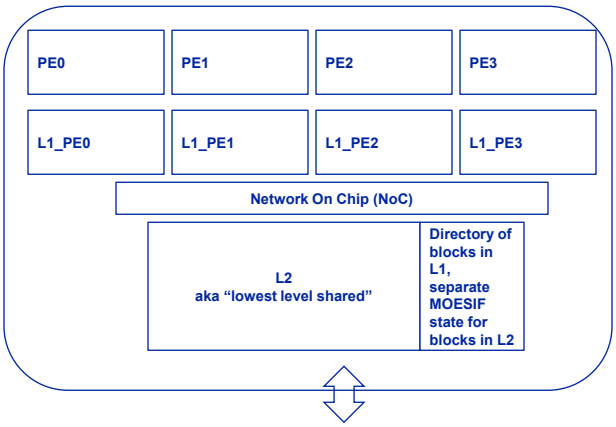
External M(O)(E)SI(F) snoopy bus

- Problems with this:
- Need high bandwidth off-chip
- L2 caches can have redundant information– not a good use of chip resources
- Only scales as well as any snoopy bus scheme (not well!)

MP-72

72

"Last level shared"

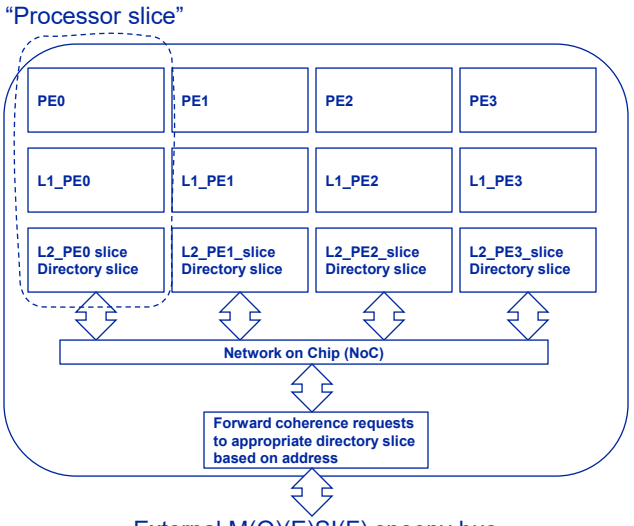


- Because existing multi-cores are built as "drop-in" replacements for uni-cores, they must maintain a glueless MP bus protocol
- To keep internal caches coherent, use a *shared cache directory* scheme
- Problem: L2 cache becomes a bottleneck

MP-73

73

LLS: Non-Uniform Cache Access



- L2 cache is LLS, but it is "sliced" between PEs (and so is the directory)
- L1 miss to X sends message to L2 slice in hash(X, N) where N is # of PEs (example hash is X mod N)

MP-74

74

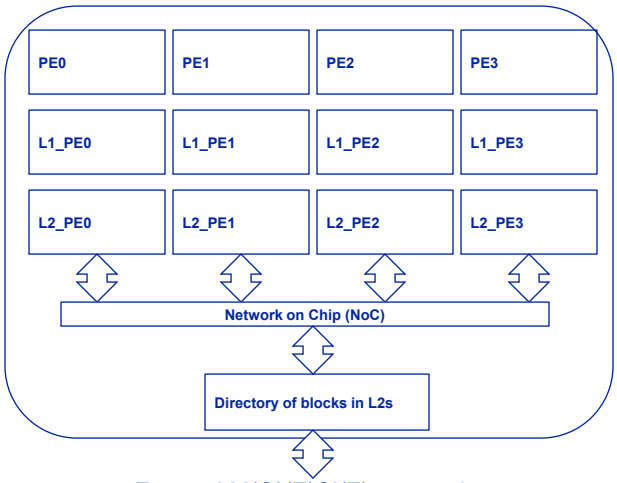
LLP vs. LLS caches

- LLS (Last Level Shared)
  - Benefits: only one copy of the data on the chip
  - Problems: LLS is either a hotspot, or it's banked (NUCA). In NUCA, L2 hit times can vary widely (data in wrong corner of chip)
- LLP (Last Level Private)
  - Benefits: No "data in the wrong corner of the chip" problem
  - Problems: Multiple copies of data is wasteful.

MP-75

75

Hybrid: LLP + shared directory



•L2 cache is LLP, but **directory is shared**

MP-76

76

Coherence hierarchies

- Often sharing is limited to "within a domain,"
  - Domain = chip, board, cabinet, etc
- Programming paradigms encourage this
  - GPU programming and CUDA/OpenGL
  - Partitioned Global Address Space model in many parallel languages
- Solution: need *hierarchies* of coherence protocols
- Problems with this:
  - Composing multiple coherence protocols makes validation harder
  - No standard way to interface hierarchies across different vendors

MP-77

77

Summary: coherence tool kit

- Snooping
  - Has limited scalability because requires a bus
- Memory based (or shared cache based) directories:
  - Reducing overhead using limited pointer schemes
  - Still have scaling issues
- Cache-based / linked-list schemes
  - Scale well
  - Invalidates take many transactions to proceed
- Likely Hybrid: hierarchy of schemes

MP-85

85